

HMSpectralGun User Manual

Turbospectrum-based stellar spectral synthesis

Updated LaTeX edition

Project: HMSpectralGun
Scope: Serial and parallel synthetic-spectrum production
Status: Operational manual
Language: English

May 28, 2026

Contents

1 Purpose and Scope	2
2 Quick Start	2
3 Requirements	2
3.1 Python environment	2
3.2 Turbospectrum runtime	3
4 Main Input File: <code>input.ts</code>	3
4.1 Required structure	3
4.2 Table columns	4
5 Auxiliary Input Files	5
5.1 Abundance file	5
5.2 Line-list index file	5
6 Serial Production Workflow	5
7 Parallel Production Workflow	5
8 Optional Shell Commands from Anywhere	6
9 Generated Outputs and Naming	6
10 Troubleshooting	6
11 Pre-Batch Checklist	7
A Minimal <code>input.ts</code> Template	7
B Minimal <code>abu.ts</code> Template	7
C Minimal Line-List Index Template	8

1 Purpose and Scope

HMSpectralGun generates synthetic stellar spectra using Turbospectrum. This manual covers the production workflow for synthetic spectra with the two production entry points currently documented for routine use:

Entry point	Purpose
<code>main.py</code>	Serial production. It reads an <code>input.ts</code> file and synthesizes the requested spectra one after another.
<code>main_parallel.py</code>	Parallel production. It follows the same synthesis logic, but distributes independent spectra across multiple worker processes.

Important

This manual is limited to synthetic-spectrum production. Line-by-line abundance fitting is not part of the production manual scope. If a line-fit configuration is present in the repository, treat it as a separate analysis workflow and adapt its placeholder paths to the local dataset before use.

2 Quick Start

From a fresh clone, create a Python environment, install the required packages, export the runtime paths for Turbospectrum, and run a small validation job.

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Export the minimum runtime paths:

```
export HMSPECTRALGUN_EXEC_PATH="/path/to/turbospectrum/exec-gf"
export HMSPECTRALGUN_LAUNCH_PATH="/path/to/turbospectrum/COM/santerre"
```

Prepare `input.ts` and run the serial workflow:

```
python main.py --input input.ts --progress
```

For a parallel batch:

```
python main_parallel.py --input input.ts
```

Note

The recommended first run is a single-spectrum test. Do not start a large batch until one spectrum has been generated, inspected, and matched to the expected wavelength range, resolution, abundance pattern, and output naming convention.

3 Requirements

3.1 Python environment

Python 3.10 or newer is recommended. The production scripts use the following Python packages:

- `numpy`, `pandas`, `tqdm`, and `mendeleev`;

- `scipy`, for resampling and numerical utilities;
- `PyAstronomy`, for instrumental convolution;
- `matplotlib`, when plotting or report utilities are available in the local workflow.

A minimal import check is:

```
python -c "import numpy, pandas, scipy, matplotlib; print('core stack: ok')"
```

```
python -c "import mendelev, PyAstronomy; print('spectral stack: ok')"
```

3.2 Turbospectrum runtime

Turbospectrum must be compiled and reachable through environment variables. The recommended variables are:

Variable	Meaning
HMSPECTRALGUN_EXEC_PATH	Directory containing the Turbospectrum executables, typically including <code>babsma_lu</code> and <code>bsyn_lu</code> .
HMSPECTRALGUN_LAUNCH_PATH	Directory where generated <code>.com</code> scripts are written and executed.

Optional overrides are available for non-standard installations:

Variable	Default or use case
HMSPECTRALGUN_DATASET_MODEL_PATH	MARCS model dataset path. Default: <code>./marcs_generator/dataset</code> .
HMSPECTRALGUN_CONTOPAC_PATH	Continuum opacity path. Default: <code>\${HMSPECTRALGUN_LAUNCH_PATH}/contopac</code> .
HMSPECTRALGUN_INTERPOLATOR_EXE	MARCS interpolation executable. Default: <code>./marcs_generator/interpol_modeles</code> .
HMSPECTRALGUN_BABSMASMA	Explicit path to <code>babsma_lu</code> , if a directory-level executable path is not sufficient.
HMSPECTRALGUN_BSYN	Explicit path to <code>bsyn_lu</code> , if a directory-level executable path is not sufficient.

Important

On Linux, the generated Turbospectrum `.com` scripts use `csh` with the shebang `#!/bin/csh -f`. Install `csh` or `tcsh` before running production jobs on systems where it is not already available.

4 Main Input File: `input.ts`

The serial and parallel workflows are driven by an `input.ts` file. The parser expects six header lines followed by a spectral-production table.

4.1 Required structure

1. `savepath`: output directory for generated spectra;
2. `linelistpath`: directory containing line-list files and line-list index files;

3. `modelpath`: directory containing atmosphere models or model products;
4. `ExplicitModel=True|False`;
5. `interp=True|False|nearest`;
6. third keyword, read by the parser as `NLTE`;
7. spectral-production table with either 15 or 17 columns.

Minimal example:

```
/path/output_spectra/
/path/linelists/
/path/models/
ExplicitModel=False
interp=True
NLTE=False
3600,1.0 -1.00 0.20 15000 15500 2.0 st * 28000 0.02 * linelistH.ts abu.ts
* txt
```

4.2 Table columns

No.	Column	Meaning
1	<code>Model</code>	Explicit model filename, or compact atmospheric identifier <code>Teff,logg</code> .
2	<code>[Fe/H]</code>	Metallicity.
3	<code>[a/Fe]</code>	Alpha-enhancement parameter.
4	<code>lam_i</code>	Initial wavelength.
5	<code>lam_f</code>	Final wavelength.
6	<code>xi</code>	Microturbulence.
7	<code>chemistry</code>	Chemistry mode: <code>st</code> , <code>ap</code> , <code>ae</code> , <code>mc</code> , or <code>hc</code> .
8	<code>sampl</code>	Resampling step, or <code>*</code> to skip resampling.
9	<code>RES</code>	Resolving power for instrumental convolution.
10	<code>resnum</code>	Numerical synthesis step used by Turbospectrum.
11	<code>monoelem</code>	<code>*</code> , an atomic species such as <code>FeI</code> , or a molecule such as <code>CO</code> , <code>OH</code> , or <code>CN</code> .
12	<code>linelist_file</code>	Index file listing the line-list files to use.
13	<code>abu_file</code>	Abundance file.
14	<code>snr</code>	Signal-to-noise value for noise injection, or <code>*</code> .
15	<code>extension</code>	Output extension, for example <code>txt</code> .
16	<code>override_elem</code>	Optional atomic number for direct abundance override.
17	<code>override_xfe</code>	Optional forced <code>[X/Fe]</code> value.

Important

If columns 16 and 17 are present, the explicit abundance override has maximum priority over the abundance value read from `abu_file`.

5 Auxiliary Input Files

5.1 Abundance file

The abundance file, for example `abu.ts`, is a two-column text file:

- column 1: atomic number;
- column 2: abundance offset $[X/Fe]$.

Example:

```
8 0.20
12 0.20
26 0.00
612613 15.0
```

The special identifier 612613 is interpreted as the carbon isotopic ratio C12/C13.

5.2 Line-list index file

The value given in `linelist_file` is an index file containing one line-list filename per line:

```
vald_1.lin
vald_2.lin
```

Files are resolved relative to `linelistpath` for the production workflow.

6 Serial Production Workflow

The serial workflow is the safest option for validation, debugging, and small jobs.

```
python main.py --input input.ts
```

With a progress bar:

```
python main.py --input input.ts --progress
```

The workflow performs the following operations:

1. reads and validates the production rows from `input.ts`;
2. selects, prepares, or interpolates the atmospheric model according to `ExplicitModel` and `interp`;
3. writes the Turbospectrum `.com` script into the launch directory;
4. runs Turbospectrum;
5. creates a descriptive header in the output spectrum;
6. optionally applies instrumental convolution using `RES`;
7. optionally applies resampling using `sampl`;
8. optionally injects noise using `snr`.

7 Parallel Production Workflow

The parallel workflow is intended for batch production after the serial validation run has succeeded.

```
python main_parallel.py --input input.ts
```

Main characteristics:

- uses multiple processes, typically up to `cpu_count() - 1`;

- writes a main log named `synth_main_YYYYMMDD_HHMMSS.log`;
- creates dedicated worker logs;
- renames final outputs with a suffix such as `_k<index>` to avoid collisions between workers.

Use the parallel workflow when the input file contains many independent spectra, the machine has enough CPU cores, and the serial workflow has already validated the configuration.

8 Optional Shell Commands from Anywhere

To launch the production workflows without manually moving into the repository directory, define shell helper functions.

For `zsh`, add the following block to `~/.zshrc`. For `bash`, add the same block to `~/.bashrc`.

```
spectralgun() {
  /path/to/HMSpectralGun/.venv/bin/python /path/to/HMSpectralGun/main.py --input "${1:-
  input.ts}" --progress
}

parallel_spectralgun() {
  /path/to/HMSpectralGun/.venv/bin/python /path/to/HMSpectralGun/main_parallel.py --
  input "${1:-input.ts}"
}
```

Reload the shell configuration:

```
source ~/.zshrc
# or, if using bash:
source ~/.bashrc
```

Usage:

```
spectralgun /path/to/input.ts
parallel_spectralgun /path/to/input.ts
```

9 Generated Outputs and Naming

The production scripts generate Turbospectrum launch files, synthetic spectra, logs, and optional post-processed products.

Product	Location or convention
Turbospectrum scripts	<name>.com files written in HMSPECTRALGUN_LAUNCH_PATH.
Synthetic spectra	Written to <code>savepath</code> .
Convolved spectra	Typically include a resolution tag such as <code>_RXXk</code> .
Noise-injected spectra	Typically include a tag such as <code>_SNR<value></code> .
Abundance-overridden spectra	Include an abundance tag such as <code>_Fep010</code> .
Parallel main log	<code>synth_main_YYYYMMDD_HHMMSS.log</code> .
Worker logs	Dedicated log files generated by the parallel workflow.

10 Troubleshooting

Symptom	Likely cause and action
ModuleNotFoundError	A Python dependency is missing or the wrong environment is active. Activate the intended environment and install the missing package.
No safe interpolation pair	The MARCS grid does not contain a safe interpolation pair for the requested parameters. Check the model dataset coverage.
No complete safe MARCS interpolation cube	The local model dataset is incomplete for the requested interpolation. Expand or correct the MARCS grid.
Missing/empty spectrum output	Turbospectrum did not produce a valid spectrum. Inspect the generated <code>.com</code> file and logs in the launch directory.
Spectrum already exists	A non-empty output already exists. The run was skipped intentionally to avoid overwriting existing products.

11 Pre-Batch Checklist

Checklist

Before launching a long batch, verify the following points:

- the Python environment is active and all required imports succeed;
- `HMSPECTRALGUN_EXEC_PATH` points to the Turbospectrum executable directory;
- `HMSPECTRALGUN_LAUNCH_PATH` exists and is writable;
- `savepath`, `modelpath`, and `linelistpath` exist and are readable or writable as needed;
- `input.ts` has been tested on one spectrum;
- all line-list index files resolve to existing line lists;
- all `abu_file` entries are present and correctly formatted;
- `csh` or `tcsh` is installed on Linux systems;
- existing output files have been intentionally kept, moved, or removed before rerunning.

A Minimal `input.ts` Template

```
/path/output_spectra/
/path/linelists/
/path/models/
ExplicitModel=False
interp=True
NLTE=False
3600,1.0 -1.00 0.20 15000 15500 2.0 st * 28000 0.02 * linelistH.ts abu.ts
* txt
```

B Minimal `abu.ts` Template


```
8 0.20  
12 0.20  
26 0.00  
612613 15.0
```

C Minimal Line-List Index Template

```
vald_1.lin  
vald_2.lin
```